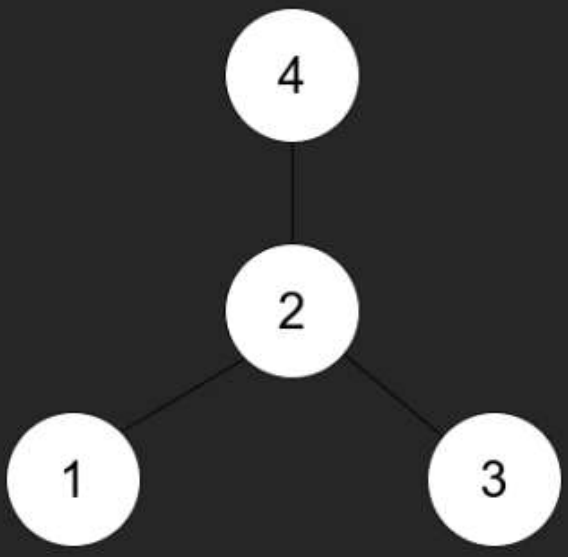
# 1791. Find Center of Star Graph

Easy | Topics | Companies | Hint

There is an undirected **star** graph consisting of `n` nodes labeled from `1` to `n`. A star graph is a graph where there is one **center** node and **exactly** `n - 1` edges that connect the center node with every other node.

You are given a 2D integer array `edges` where each `edges[i] = [u_i, v_i]` indicates that there is an edge between the nodes `u_i` and `v_i`. Return the center of the given star graph.

**Example 1:**




```
Input: edges = [[1,2],[2,3],[4,2]]
Output: 2
Explanation: As shown in the figure above, node 2 is connected to every other node, so 2 is the center.
```

**Example 2:**

> **Input:** edges = [[1,2],[5,1],[1,3],[1,4]]
> **Output:** 1

**Constraints:**

- `3 <= n <= 10^5`
- `edges.length == n - 1`
- `edges[i].length == 2`
- `1 <= u_i, v_i <= n`
- `u_i != v_i`
- The given `edges` represent a valid star graph.

Qs / Day-91 / 7-11-26.

Leetcode 1791 - Find center of Star Graph.

We r given: n nodes (vertices), numbered from 1 to n.

→ n-1 edges (connections)

→ every edge connects two nodes

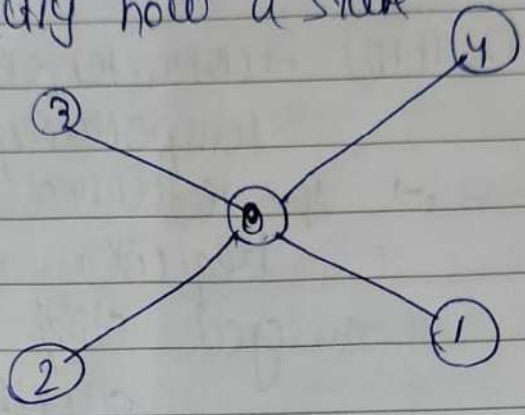
→ there is exactly one center node that is directly connected to every other node.

Our task is to find & return that center node.

what is a stare Graph?

⇒ imagine the sun & planets.
- the sun is on the centre
- every planet is connected directly to the sun.
- the planets are not necessarily connected to each other.

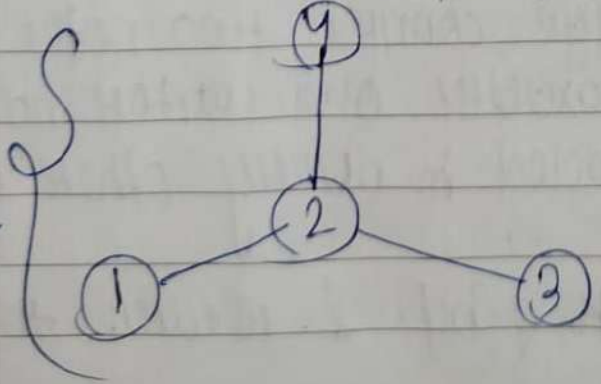
that's exactly how a stare graph works.




→ the centre node has connections to all other nodes → the centre node has degree n-1.
degree means the no. of edges connected to a particular node.

Example 1 → input edges = {{1,2},{2,3},{4,2}};

node 1 is connected to node 2




visual representation

here, node 2 is connected to:-
node 1, node 3, node 4,

therefore, output = 2

Optimal approach =

In a star graph, every edge MUST contain the center node.

Because the problem guarantees that:-

↳ there is exactly one center

↳ the center is connected to every other node

↳ every edge connects the center to another node.

So any two edges must share the center node, we don't need to examine all the edges, we just need the first two.

```
class Solution{ public:
Return type int → int findCenter(vector<vector<int>>& edges){
    if (edges[0][0] == edges[1][0] ||
        edges[0][0] == edges[1][1]){
        return edges[0][0];
    }
    return edges[0][1]; }};
```

vector<int>: a 1D vector of integers

vector<vector<int>>: a 2D vector (a vector containing other vectors)

edges = the variable name.

& = pass by reference.

our input looks like this,

vector<vector<int>> edges = {

{1, 2},
{2, 3},
{4, 2}

};

→ each inner vector represents one edge connecting two nodes,

why & → without &,

a copy of the entire vector would be passed to the function,

with & → ~~vector~~ the function directly accesses the original vector without making a copy.

this saves memory & avoids unnecessary copy, we are only reading the edges, so we could also use,

(const vector<vector<int>>& edges)

↳ this additionally prevents accidental modification.

Our input :-

edges = {{1,2}, {2,3}, {4,2}};

Remember that a 2D vector uses two indices.
1st index = which edge
2nd index = which node inside that edge,
Therefore :-

| | value | meaning |
|---|---|---|
| edges[0][0] | 1 | 1st node of 1st edge |
| edges[0][1] | 2 | 2nd " " 1st edge |
| edges[1][0] | 2 | 1st " " 2nd edge |
| edges[1][1] | 3 | 2nd " " " " |

now, our condition becomes,

if (1 == 2 || 1 == 3)

both comparisons are false, so we must avoid the if block, but why are we checking only edges[0][0]? → bcz the center must be one of the two endpoints of the 1st edge.

We 1st check whether its first endpoint appears in the 2nd edge,

then, return edges[0][0]; →
if the condition is true, it means the 1st endpoint of the 1st edge is also present in the 2nd edge, therefore it is the common node & hence the center.

Any test:

edges = {{1,2}, {5,1}, {1,3}, {1,4}};

Step-1
```
edges[0][0] = 1;
edges[0][1] = 2;
edges[1][0] = 5;
edges[1][1] = 1;
```

Step-2
```
if (1 == 5 || 1 == 1)
    Java || then = true;
```

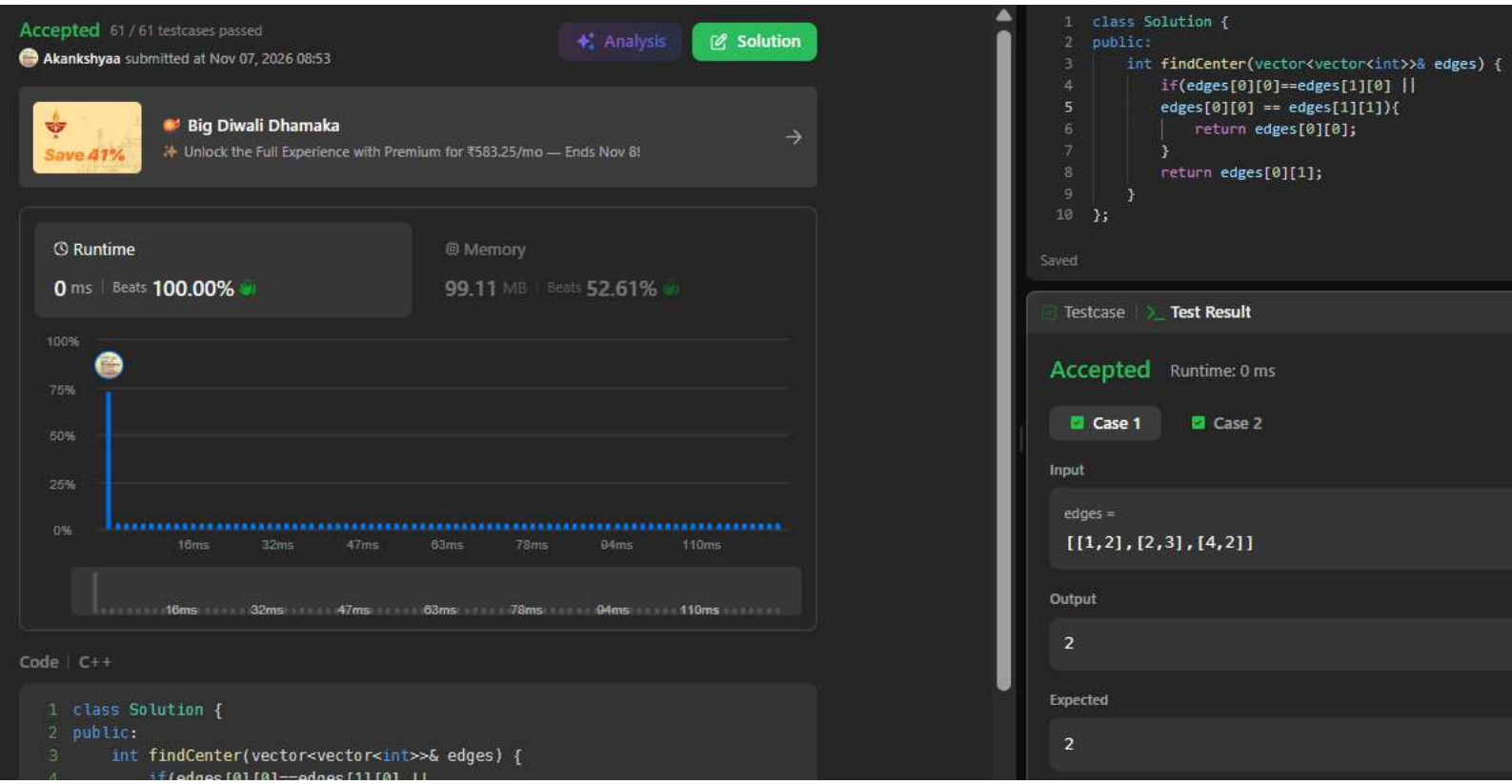
Step-3
```
return edges[c][c];
```
which means:- return 1 } output.

T.C = O(1) → we only access 1st two, we don't run any loops, we don't traverse all the edges, regardless of whether there are 3 nodes are 10,000 nodes, we perform a const. no. of operations.

S.C = O(1)

→ we don't create an additional array.
→ we don't create a graph adjacency list.
→ we don't use recursion.
→ we only use a few comparisons & return a value.

Anyhow we perform 100,000 operations when just 2 edges are enough? → that's the beauty of algorithm optimization.


Accepted 61 / 61 testcases passed
Akankshyaa submitted at Nov 07, 2026 08:53
Analysis
Solution
Save 41%
Big Diwali Dhamaka
Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!
Runtime
0 ms | Beats 100.00%
Memory
99.11 MB | Beats 52.61%
100%
75%
50%
25%
0%
16ms
32ms
47ms
63ms
78ms
94ms
110ms
Code | C++
class Solution {
public:
int findCenter(vector<vector<int>>& edges) {
class Solution {
public:
int findCenter(vector<vector<int>>& edges) {
if(edges[0][0]==edges[1][0] ||
edges[0][0] == edges[1][1]){
return edges[0][0];
}
return edges[0][1];
}
};
Saved
Testcase | Test Result
Accepted Runtime: 0 ms
Case 1
Case 2
Input
edges =
[[1,2],[2,3],[4,2]]
Output
2
Expected
2